Microbenchmarks: Measuring algorithm run time

Peter Sestoft
IT University of Copenhagen

KCS Practical Concurrent and Parallel Programming
Monday 2026-10-19 *

Plan for today

- Performance measurements
- A class for measuring elapsed wall-clock time
  - Mark0-5: Towards reliable measurements
  - Mark6-7: Automated general measurements
  - Mark8, Mark8Setup: Scalability measurements e.g. for sorting
- Platform identification
- Aside: New virtual threads and old platform threads
- Measuring thread creation etc
- Measuring cache and memory latency
- General benchmarking advice, warnings and pitfalls

Reading materials week08 to week10

- Week08 (this week)
  - Read *Microbenchmarks in Java, C# and C*, sections 1-4, 7, 9-11
  - Lecture notes section 9.1: *Platform threads, virtual threads and …*
- Week09
  - Lecture notes chapter 8: *The computer's CPU and RAM*
    - Required reading, but there will not be exam questions exclusively on this
  - Lecture notes section 9.1: *Platform threads, virtual threads and …*
  - Lecture notes section 9.2: *Long adder scalability case study*
  - (Lecture notes section 9.3: *Concurrent hash map scalability case …)*
- Week10
  - Chapter 24 of Java Precisely: *Streams for Bulk Data*
  - Lecture notes section 9.4: *Parallel k-means clustering and cache …*

How long does this method take?

```
private static double multiply(long i) {
  double x = 1.1 * (double)(i & 0xFF);
  return x * x * x * x * x * x * x * x * x * x
       * x * x * x * x * x * x * x * x * x * x;
}
```

- Do a long operation, long-double conversion, and 20 floating-point multiplications
- This takes maybe 20 * 0.25 = 5 ns
- Tricks used in this code:
  - Make result depend on `i` to avoid caching
  - The `i & 0xFF` is in range 0—255, avoids overflow
  - Multiply `i & 0xFF` by 1.1 to make it floating-point

Back-of-the envelope calculations

- 4 GHz processor = 0.25 ns/cycle = 0.25×10^{-9} s
- Throughput:
  - Addition or multiplication takes <= 1 cycle
  - Division maybe 10 cycles
  - Transcendental functions, `sin(x)` maybe 50?
- Instruction-level parallelism
  - 2-4 arithmetic operations/cycle, sometimes
- Memory latency
  - Registers: 1 cycle
  - L1 cache: a few cycles
  - L2 cache: many cycles
  - RAM: hundreds of cycles – expensive cache misses

A simple Timer class for Java

- We measure elapsed wall-clock time
  - This is what matters in reality
  - Can measure uniformly on Linux, MacOS, Windows
  - Enables comparison Java, C#, C, F#, Scala, Rust etc

Benchmark.java

```
public class Timer {
  private long start, spent = 0;
  public Timer() { play(); }
  public double check()
  { return (System.nanoTime()-start+spent)/1e9; }
  public void pause() { spent += System.nanoTime()-start; }
  public void play() { start = System.nanoTime(); }
}
```

- Alternatives: total CPU time, or user + kernel
- Never use imprecise and slow **`new Date().getTime()`**
- Q: Reasons to measure CPU time instead?

Mark0: naïve attempt

```
public static void Mark0() {
  Timer t = new Timer();
  double dummy = multiply(10);
  double time = t.check() * 1e9;
  System.out.printf("%6.1f ns%n", time);
}
```

Useless

- Useless because
  - Runtime start-up costs larger than execution time
  - Timer resolution too coarse, maybe 100 ns
  - Results are unrealistic and variable:

```
2375.0 ns
2250.0 ns
2916.0 ns
```

Mark1: Measure many operations

```
public static void Mark1() {
  Timer t = new Timer();
  long count = 1_000_000;
  for (long i=0; i<count; i++) {
    double dummy = multiply(i);
  }
  double time = t.check() * 1e9 / count;
  System.out.printf("%6.1f ns%n", time);
}
```

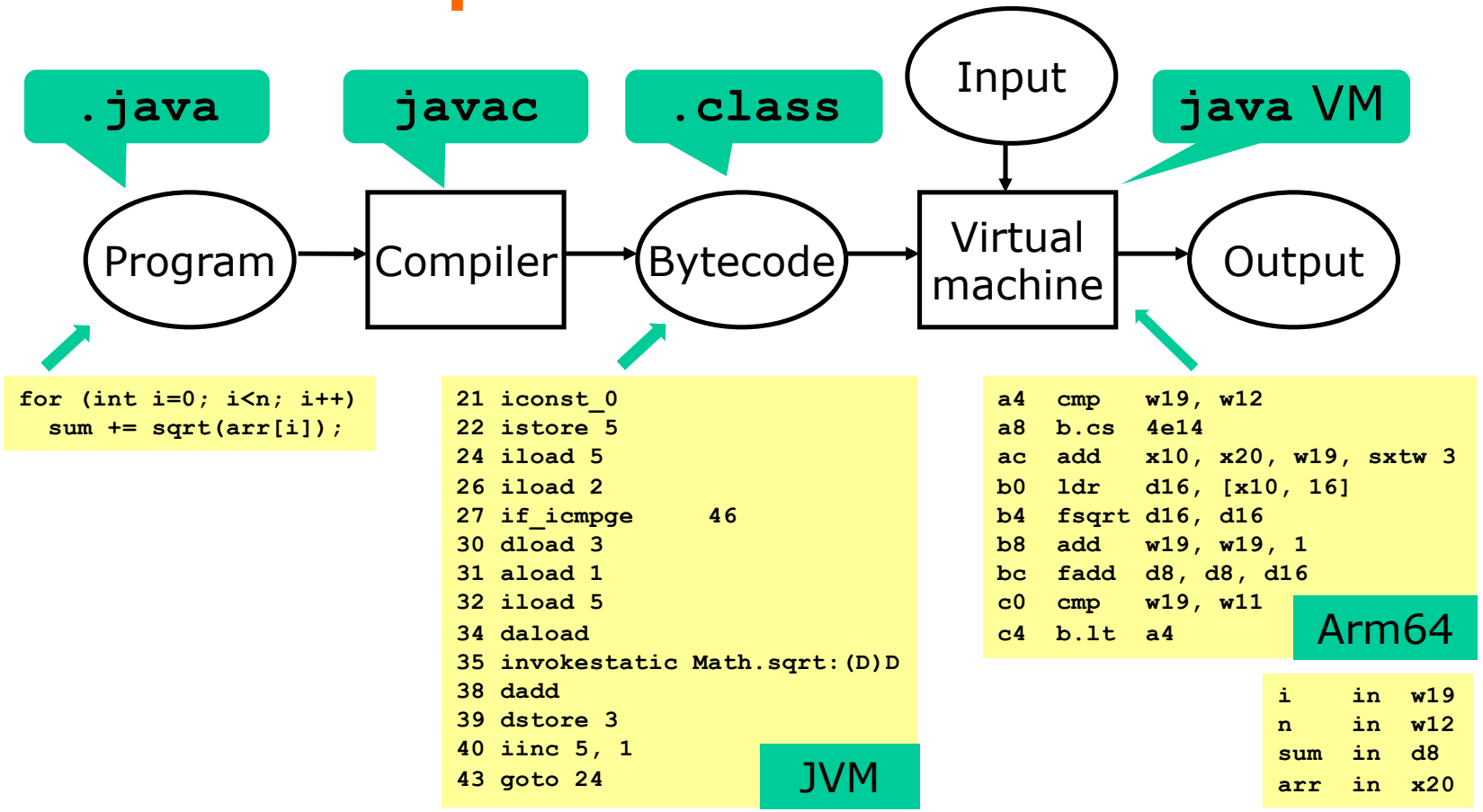
Quite useless

```
4.0 ns
3.9 ns
4.4 ns
```

- Measure 1 million calls; better but fragile:
  - If **`count`** is larger, optimizer may notice that result of **`multiply`** is not used, and remove call
  - So-called "dead code elimination"
  - May give completely unrealistic results

```
0.3 ns
0.3 ns
0.3 ns
```

Java compiler and virtual machine



- The **javac** compiler is simple, makes no optimizations
- The **java** runtime system (JIT) is clever, makes many

Mark2: Avoid dead code elimination

```
public static double Mark2() {
  Timer t = new Timer();
  long count = 100_000_000;
  double dummy = 0.0;
  for (long i=0; i<count; i++)
    dummy += multiply(i);
  double time = t.check() * 1e9 / count;
  System.out.printf("%6.1f ns%n", time);
  return dummy;
}
```

```
1.9 ns
1.9 ns
1.9 ns
```

- Much more reliable (though numbers low)

Mark3: Automate multiple samples

```
long n = 10;
long count = 100_000_000;
double dummy = 0.0;
for (long j=0; j<n; j++) {
  Timer t = new Timer();
  for (long i=0; i<count; i++)
    dummy += multiply(i);
  double time = t.check() * 1e9 / count;
  System.out.printf("%6.1f ns%n", time);
}
```

Number of samples

Iterations per sample

```
1.9 ns
1.9 ns
1.8 ns
1.8 ns
1.8 ns
1.9 ns
1.8 ns
1.8 ns
1.8 ns
1.8 ns
```

- Multiple samples gives an impression of reproducibility

Mark4: Compute standard deviation

Benchmark.java

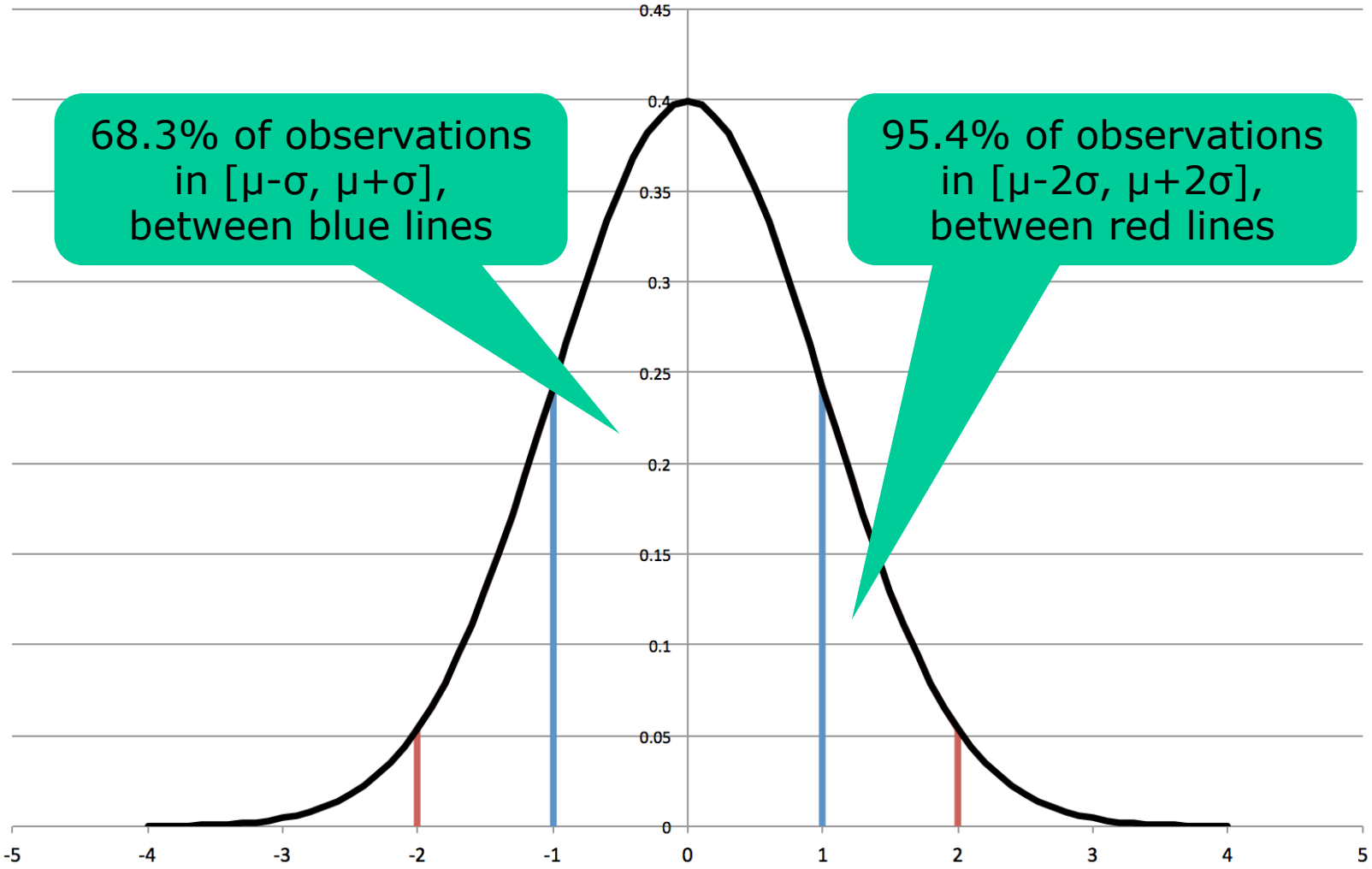
```
long count = 100_000_000;
double dummy = 0.0;
double st = 0.0, sst = 0.0;
for (long j=0; j<n; j++) {
  Timer t = new Timer();
  for (long i=0; i<count; i++)
    dummy += multiply(i);
  double time = t.check() * 1e9 / count;
  st += time; sst += time * time;
}
double mean = st/n,
       sdev = Math.sqrt((sst - mean*mean*n)/(n-1));
System.out.printf("%6.1f ns +/- %6.3f%n", mean, sdev);
```

Is this a reasonable iteration count?

- The standard deviation summarizes the variation around the mean, in a single number

```
1.8 ns +/-  0.023
```

The normal distribution $N(\mu,\sigma^2)$



Mark5: Auto-choose iteration count

Benchmark.java

```
long n = 10, count = 1, totalCount = 0;
double dummy = 0.0, runningTime = 0.0, st = 0.0, sst = 0.0;
do {
  count *= 2;
  st = sst = 0.0;
  for (long j=0; j<n; j++) {
    Timer t = new Timer();
    for (long i=0; i<count; i++)
      dummy += multiply(i);
    runningTime = t.check();
    double time = runningTime * 1e9 / count;
    st += time;
    sst += time * time;
    totalCount += count;
  }
  double mean = st/n, sdev = Math.sqrt(...);
  System.out.printf("%6.1f ns ... ", mean, sdev, count);
} while (runningTime < 0.25);
return dummy / totalCount;
```

Double `count` until ...

... loop runs at least 0.25 sec

Example results from Mark5

mean time	sdev	count
239.5 ns +/-	457.91	2
83.4 ns +/-	30.29	4
75.5 ns +/-	19.51	8
90.6 ns +/-	44.45	16
76.2 ns +/-	27.74	32
66.1 ns +/-	8.95	64
60.2 ns +/-	12.24	128
55.3 ns +/-	2.02	256
54.1 ns +/-	0.25	512
54.2 ns +/-	0.53	1024
33.0 ns +/-	14.74	2048
19.4 ns +/-	0.19	4096
4.1 ns +/-	0.21	8192
4.1 ns +/-	0.17	16384
...		
1.9 ns +/-	0.21	33554432
1.8 ns +/-	0.01	67108864
1.9 ns +/-	0.01	134217728

Outlier, maybe JIT optimization?

Good convergence

Advantages of Mark5

- The early rounds (2, 4, ...) serve as warm-up
  - Make sure the code is in memory and cache
- Measured code loop runs at least 0.25 sec
  - Roughly 500 million CPU cycles
  - Lessen impact of other activity on computer
  - Makes sure code has been JIT compiled
- Still, total time spent measuring at most 1 sec
  - Because last measurement runs at most 0.5 sec
  - and sum of previous times is same time as last one
    - because $2 + 4 + 8 + \ldots + 2^n < 2^{n+1}$
- Independent of problem and hardware

Development of the benchmarking method

- Mark0: Measure one call, useless
- Mark1: Measure many calls, nearly useless
- Mark2: Avoid dead code elimination
- Mark3: Automate multiple samples
- Mark4: Compute standard deviation
- Mark5: Automate choice of iteration count

- But need to measure other functions than **`multiply`**
- Also slower ones:
  microseconds (us) or milliseconds (ms)

Mark6: Generalize to any function

```
public interface LongToDoubleFunction {
  double applyAsDouble(long value);
}
```

From java.util.function

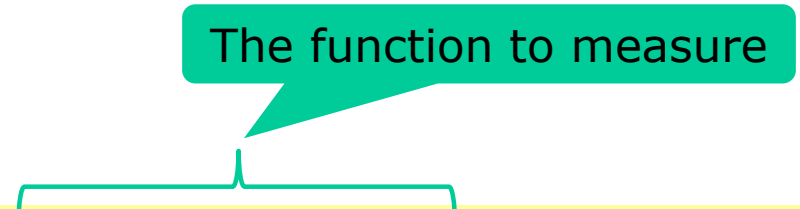
Benchmark.java

```
static double Mark6(char tunit, String msg,
                    LongToDoubleFunction f) {
  ...
  do {
    for (long j=0; j<n; j++) {
      ...
      for (long i=0; i<count; i++)
        dummy += f.applyAsDouble(i);
      double time = runningTime * scale(tunit) / count;
    }
    ...
    System.out.printf("%-25s %15.1f %cs %10.2f %10d%n", ...);
 } while (runningTime < 0.25);
  return dummy / totalCount;
}
```

Call given function `f`

Time scale: ns, us, ms

Example use of Mark6




```
Mark6('n', "multiply", i -> multiply(i));
```

```
multiply                          275.1 ns      482.04           2
multiply                          126.1 ns       44.04           4
multiply                           90.1 ns       21.26           8
multiply                          137.0 ns       61.64          16
...
multiply                            1.8 ns        0.01     8388608
multiply                            1.8 ns        0.01    16777216
multiply                            1.8 ns        0.00    33554432
multiply                            1.8 ns        0.00    67108864
multiply                            1.8 ns        0.01   134217728
multiply                            1.8 ns        0.00   268435456
```

Mark7: Print only last measurement

```
public static double Mark7(char tunit, String msg, ...) {
  ...
  do {
    ...
  } while (runningTime<0.25);
  double mean = st/n, sdev = Math.sqrt(...);
  System.out.printf("%-25s %15.1f %cs %10.2f %10d%n", ...);
  return dummy / totalCount;
}
```

Benchmark.java

Printing moved from here to outside loop

```
Mark7('n', "pow", i -> Math.pow(10.0, 0.1 * (i & 0xFF)));
Mark7('n', "exp", i -> Math.exp(0.1 * (i & 0xFF)));
Mark7('n', "log", i -> Math.log(0.1 + 0.1 * (i & 0xFF)));
Mark7('n', "sin", i -> Math.sin(0.1 * (i & 0xFF)));
Mark7('n', "cos", i -> Math.cos(0.1 * (i & 0xFF)));
Mark7('n', "tan", i -> Math.tan(0.1 * (i & 0xFF)));
...
```

Benchmark.java

Functions to measure

Mark 7 benchmarking results for Java mathematical functions

pow	**29.1 ns**	**0.05**	**16777216**
exp	**6.6 ns**	**0.14**	**67108864**
log	**7.3 ns**	**0.05**	**67108864**
sin	**6.2 ns**	**0.01**	**67108864**
cos	**6.2 ns**	**0.04**	**67108864**
tan	**16.4 ns**	**0.11**	**16777216**
asin	**5.8 ns**	**0.01**	**67108864**
acos	**5.9 ns**	**0.03**	**67108864**
atan	**6.2 ns**	**0.02**	**67108864**

- 4.0 GHz Apple M3 pro; MacOS 26.6.2; JVM 25
- So **`sin(x)`** takes 6.2 ns x 4.0 GHz = 25, approximately

Mark8: Print run parameters

```
static double Mark8(char tunit, String msg, String info,
                    LongToDoubleFunction f) {
  ...
}
```

Benchmark.java

- Used for scalability experiments
  - Running a function with a range of parameters
  - Eg quicksort time as function of array size

```
for (int size = 100; size <= 10_000_000; size *= 2) {
  int[] intArray = SearchAndSort.fillIntArray(size);
  int successItem = (int)(0.49 * size);
  Mark8('n', "binary_search_success",
        String.format("%8d", size),
        i -> binarySearch(successItem, intArray));
}
```

New

Benchmark.java

Saving measurements to a text file

- Command line in Linux, MacOS, Windows

```
java Benchmark > benchmark-20260819.txt
```

- In Linux, MacOS get both file and console

```
java Benchmark | tee benchmark-20260819.txt
```

```
gradle -PmainClass=exercises08.Benchmark run | tee ...
```

Platform identification

Benchmark.java

```
public static void SystemInfo() {
  System.out.printf("# OS:   %s; %s; %s%n",
                    System.getProperty("os.name"),
                    System.getProperty("os.version"),
                    System.getProperty("os.arch"));
  System.out.printf("# JVM:  %s; %s%n",
                    System.getProperty("java.vendor"),
                    System.getProperty("java.version"));
  // The processor identifier works only on MS Windows:
  System.out.printf("# CPU:  %s; %d \"cores\"%n",
                    System.getenv("PROCESSOR_IDENTIFIER"),
                    Runtime.getRuntime().availableProcessors());
  java.util.Date now = new java.util.Date();
  System.out.printf("# Date: %s%n",
    new java.text.SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ssZ").format(now));
}
```

- Output information about platform and date:

```
# OS:   Mac OS X; 26.6; aarch64
# JVM:  Oracle Corporation; 25
# CPU:  null; 12 "cores"
# Date: 2026-08-04T13:32:18+0200
```

4 August 2026 at 13:32 in UTC+2h

General measurement advice

- Avoid interference from other programs
- Shut down: mail, browsers, Dropbox, MS Office, Teams, ...
- Disable logging and debugging messages
- Compile with optimizations enabled (not relevant to Java)
- Never measure inside IDEs such as VS Code
- Turn off power-savings modes
- Run on mains power, not on battery
- Lots of differences between
  - Runtime systems: IBM, Microsoft, OpenJDK, Oracle … Java VMs
  - CPUs: Intel i5, i7, i9, Xeon; AMD; Apple Arm64; Raspberry Pi Arm64

Platform threads and virtual threads

- A *platform thread* is a classical Java thread, Java 1 – Java 20
  - Each platform thread needs an underlying operating system thread
  - One cannot (and should not) create many platform threads
- A *virtual thread* (Java 21) is light-weight, creating millions OK
  - Also instance of class Thread, supports same methods

```
t1 = new Thread(() -> { lc.increment(); });
t1.start();
```

week01 TestLongCounterExperiments.java

Classic platform thread create and start

```
t1 = Thread.ofPlatform().start(() -> { lc.increment(); });
```

New: does the same

```
t1 = Thread.ofVirtual().start(() -> { lc.increment(); });
```

New: make virtual thread

Operations on platform and virtual threads

- New Thread static methods, new Thread.Builder class

	Platform threads	Virtual threads
Thread builder create	`tb = Thread.ofPlatform()`	`tb = Thread.ofVirtual()`
Thread create	`th = tb.unstarted(runn)`	`th = tb.unstarted(runn)`
Thread create	`th = new Thread(runn)`	
Thread create start	`th = tb.start(runn)`	`th = tb.start(runn)`
Thread create start		`th = Thread.startVirtualThread(runn)`
Get current thread	`Thread.currentThread()`	`Thread.currentThread()`
Thread join	`th.join()`	`th.join()`
Sleep current thread	`Thread.sleep(millis)`	`Thread.sleep(millis)`
Thread start	`th.start()`	`th.start()`
Get thread id as `long`	`th.threadId()`	`th.threadId()`
Yield to other threads	`Thread.yield()`	`Thread.yield()`

Make thread builder

Make thread

Make and start thread

Same methods!

```
Thread th = Thread.ofPlatform().start(() -> { ... });
```

Good approach

Thread kind

Started or not

Virtual threads

- A virtual thread does **not** run faster than a platform thread
- But
  - is faster to create
  - takes less memory
  - handles blocking operations (input-output, database requests) well
  - does not occupy an underlying operating system thread
- More next week…

- For now, let us measure platform and virtual threads

Cost of object creation

- First: how long to create an ordinary object?

```
class Point {
  public final int x, y;
  public Point(int x, int y) { this.x = x; this.y = y; }
}
```

```
Mark6('n', "Point creation",
      i -> {
        Point p = new Point((int)i, (int)i);
        return p.hashCode();
      });
```

TestTimeThreads.java

- Result on M3 pro: approximately 33 ns
- Q: Why return **`p.hashCode()`**?
- Computing the hash code takes 1-2 ns
  - Q: How can I know that?

Cost of platform thread creation

```
Mark6('n', "Thread pltf cre",
      i -> {
        final AtomicInteger ai = new AtomicInteger();
        Thread t = Thread.ofPlatform().unstarted(() -> {
          for (int j=0; j<1000; j++)
            ai.getAndIncrement();
        });
        return t.hashCode();
});
```

TestTimeThreads.java

Thread's work, not run, not measured

What we measure

- Takes 53 ns, only slightly more than a Point
  - A big improvement over Java 8, ca 2014
- Virtual thread: 35 ns

```
Thread t = Thread.ofVirtual().unstarted(() -> { ... });
```

Cost of platform thread create + start

```
Mark6('n', "Thread pltf cre strt",
      i -> {
        final AtomicInteger ai = new AtomicInteger();
        Thread t = Thread.ofPlatform().start(() -> {
          for (int j=0; j<1000; j++)
            ai.getAndIncrement();
        });
        return t.hashCode();
});
```

TestTimeThreads.java

Actual work, mostly not run

What we measure

- Takes 23000 ns
- Setting up a platform thread takes much work
  - Even after creating it
- Virtual thread: (hard to measure reliably)

Cost of thread create+start+run+join

```
Mark6('n', "Thread pltf cre strt join",
      i -> {
        final AtomicInteger ai = new AtomicInteger();
        Thread t = Thread.ofPlatform().start(() -> {
          for (int j=0; j<1000; j++)
            ai.getAndIncrement();
          });
        try { t.join(); }
        catch (InterruptedException exn) { }
        return t.hashCode();
});
```

TestTimeThreads.java

Actual work is done

because of join()

- Total 30400 ns; actual work is 1730 ns; overhead is 28670 ns
- Virtual threads: total 7100 ns; overhead 5270 ns; 5.4 x less

- *Never create (platform) threads for small computations*

Cost of taking a free lock

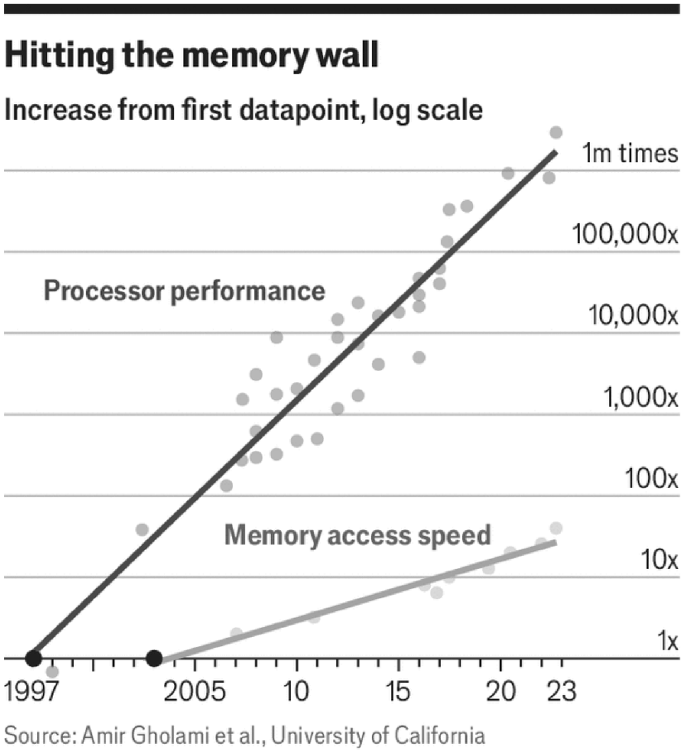
```
Mark6('n', "Uncontended lock",
      i -> {
        synchronized (obj) {
          return i;
        }
});
```

Succeeds immediately because only one thread is running

TestTimeThreads.java

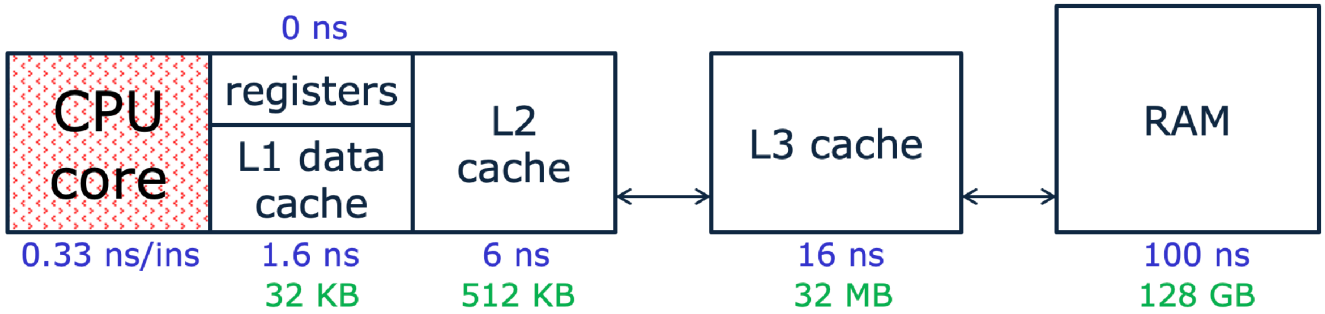
- Takes around 4 ns
- This is very fast
  - The result of much engineering on the Java runtime system
  - Was much slower in early Java
  - Do not use “double-checked-locking”, Goetz antipattern p. 349
- Q: Is it possible to measure time to take a lock already held by another thread?

CPUs have become much faster, RAM has not



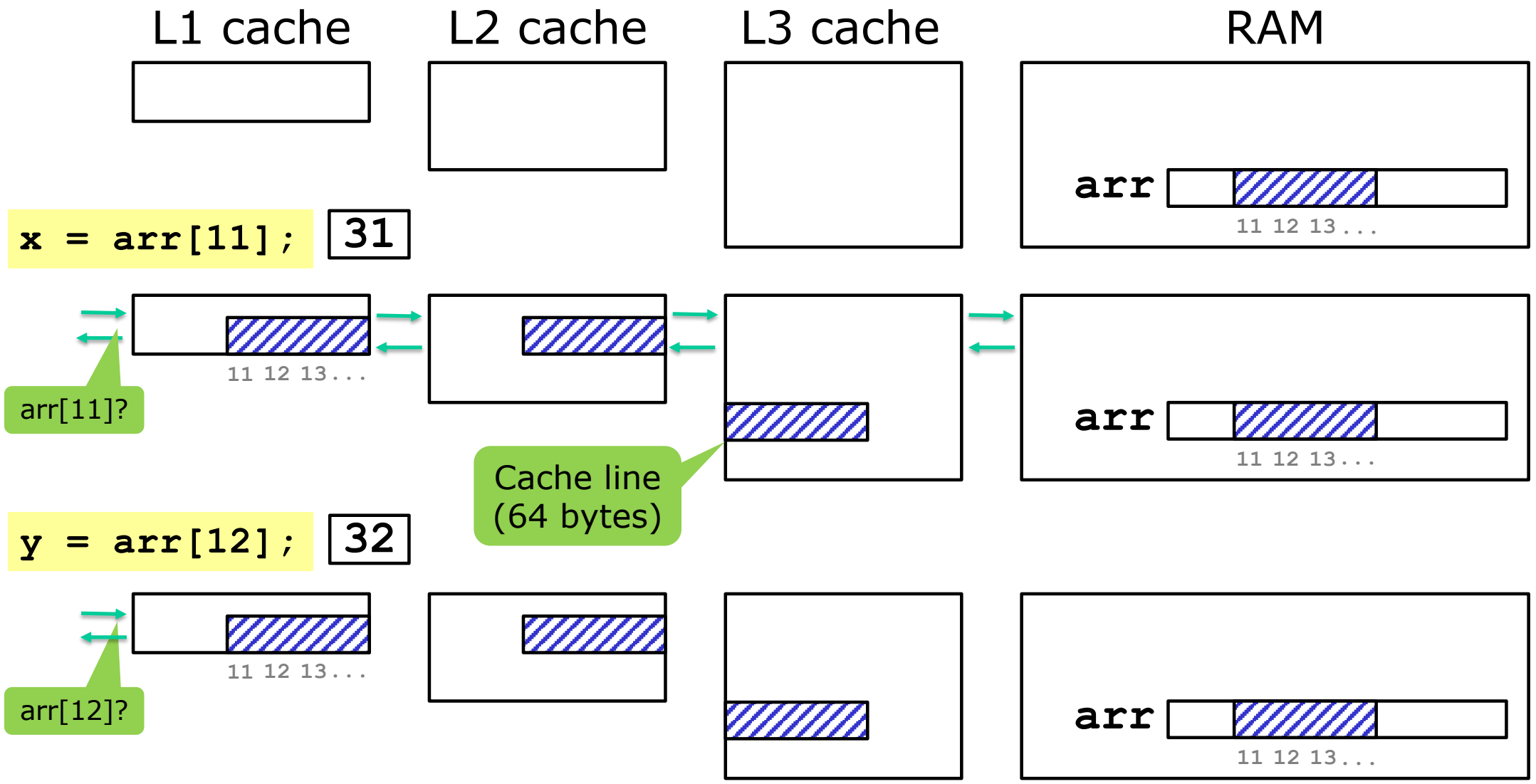
Economist Sep 2024

- A performance/price/energy trade-off
  - RAM: slow, cheap, uses little energy
  - Caches: fast, expensive, use much energy
- Caches help when there is access locality
  - Spatial locality: will access nearby data soon
  - Temporal locality: will access same data soon



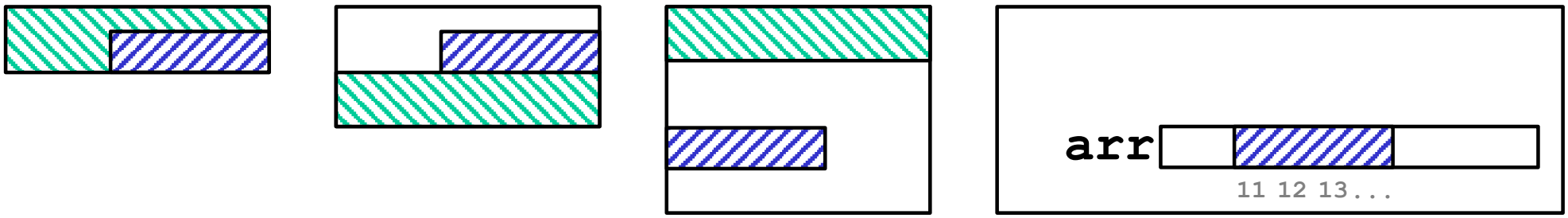
Example cache access

```
arr = { 20, 21, 22, ..., 99 }
```



When cache is full, some data will be thrown out

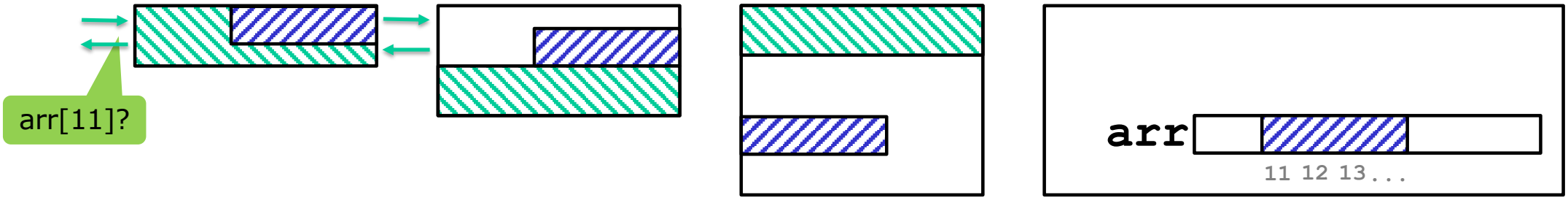
- After more data access, L1 is full, and arr[11…37] is thrown out:



- If (fast) L1 cache does not have the data, ask (slower) L2, and so on:

```
x = arr[11];
```

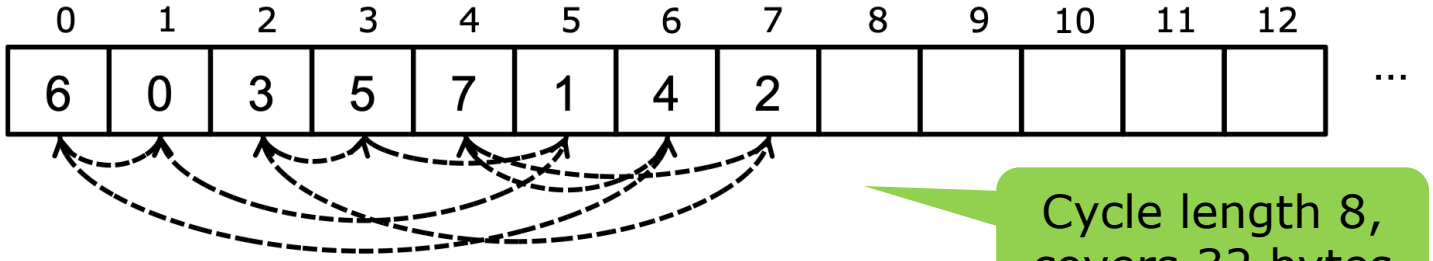
31



- Accessing a larger amount of data causes slower average access

Measuring the speed of caches and memory

- Cannot measure memory *latency* directly: too short (1-100 ns)
- Idea: Measure time to iterate through reference cycle



Cycle length 8, covers 32 bytes

Benchmark.java

- Execution time of this depends only on memory access time:

```
int k = 0;
for (int j=0; j<134_217_728; j++)
  k = arr[k];
```

Assignment takes 1 to 110 ns, depending on cache contents

Time per array access as function of KB accessed

KB accessed	ns/access
8	1.5
16	1.5
32	1.5
64	1.5
128	1.5
256	4.3
512	5.5
1,024	6.1
2,048	6.4
4,097	7.8
8,193	8.6
16,386	10.4
32,773	41.6
65,545	72.9
131,091	92.1
262,182	102.9

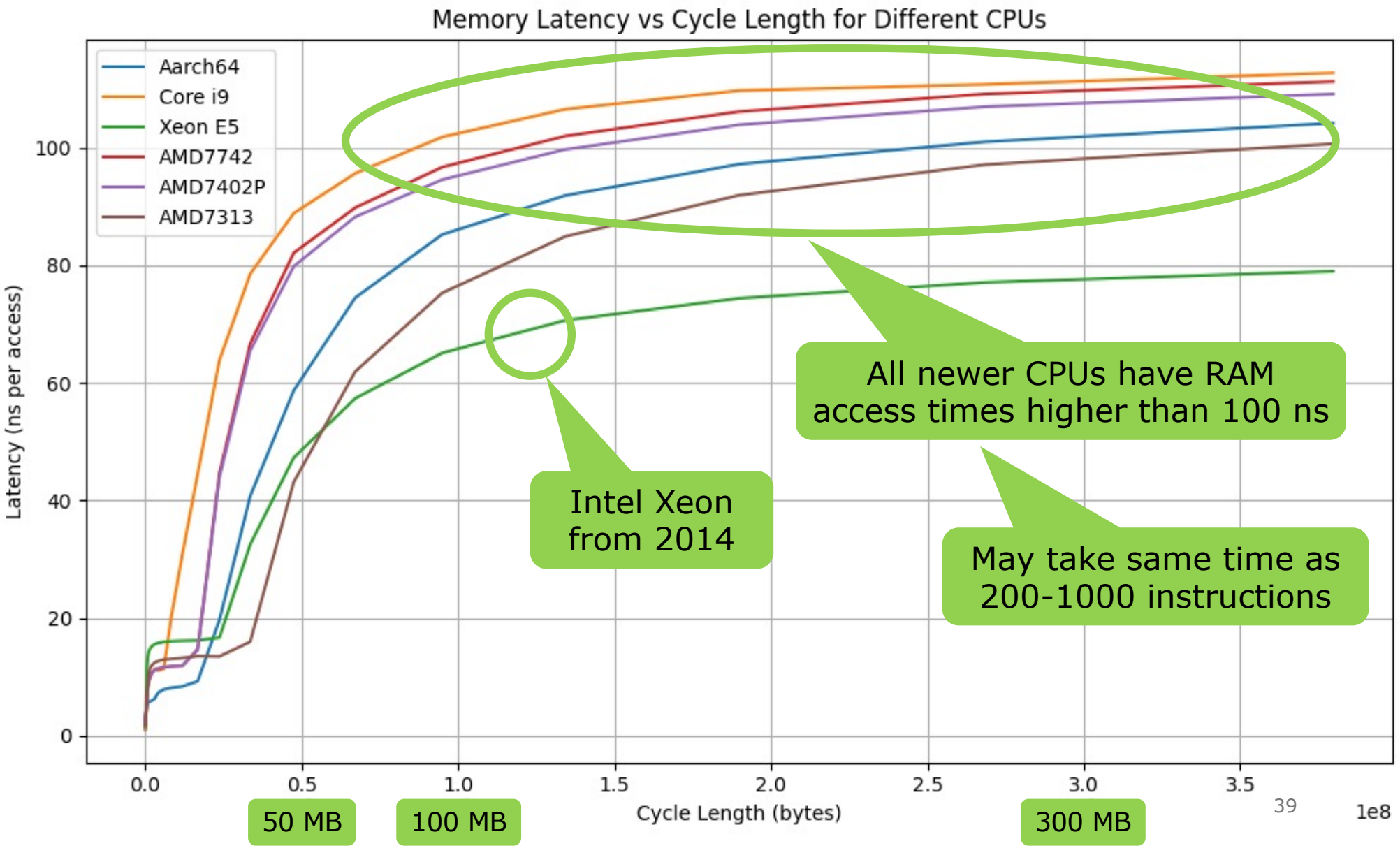
Apple M3 pro

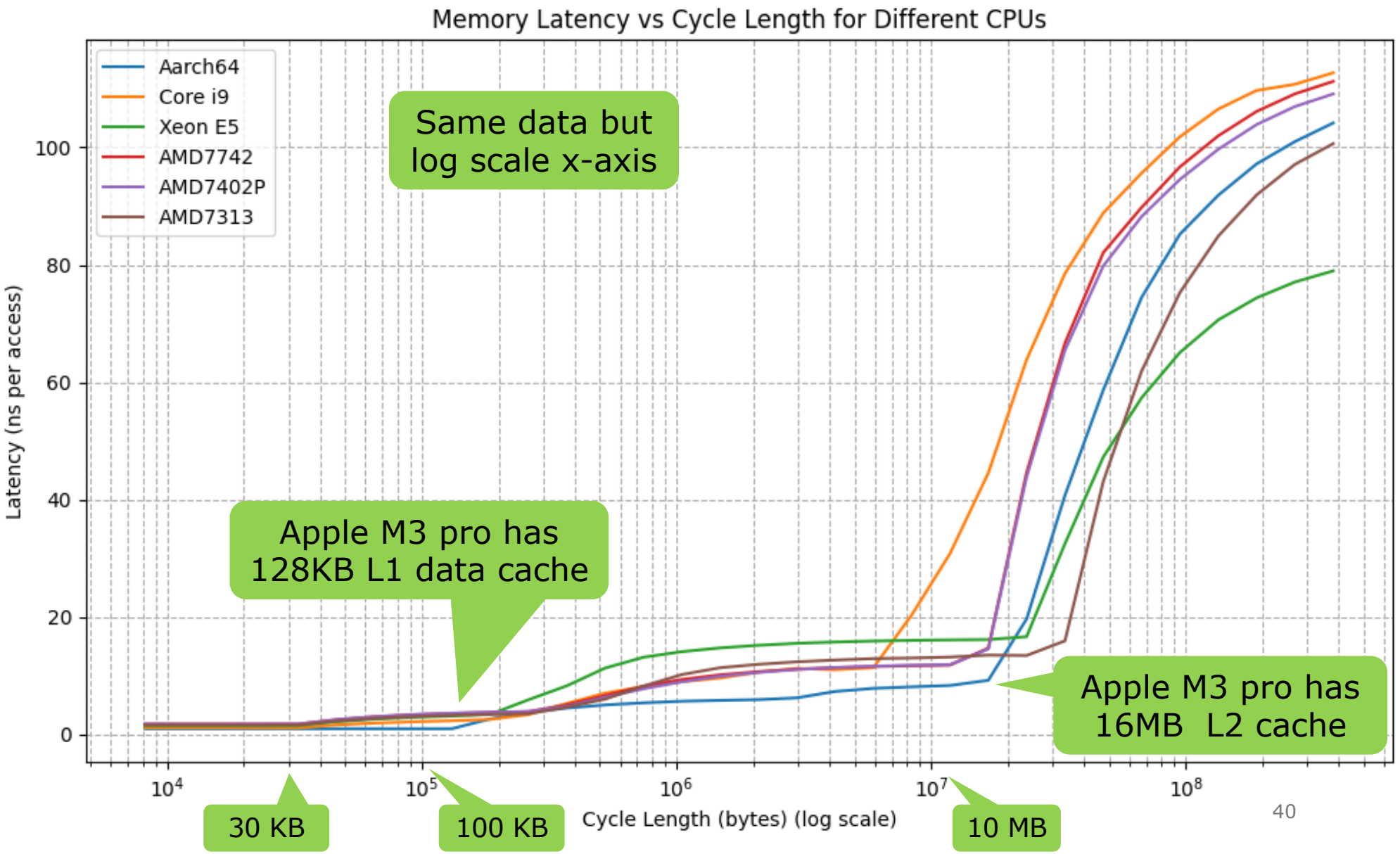
Here all accessed data fit in L1 cache

411 times slower than CPU

The reason for having caches!

benchmark.c


Memory Latency vs Cycle Length for Different CPUs
Aarch64
Core i9
Xeon E5
AMD7742
AMD7402P
AMD7313
Latency (ns per access)
0
20
40
60
80
100
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
1e8
Cycle Length (bytes)
50 MB
100 MB
300 MB
Intel Xeon from 2014
All newer CPUs have RAM access times higher than 100 ns
May take same time as 200-1000 instructions

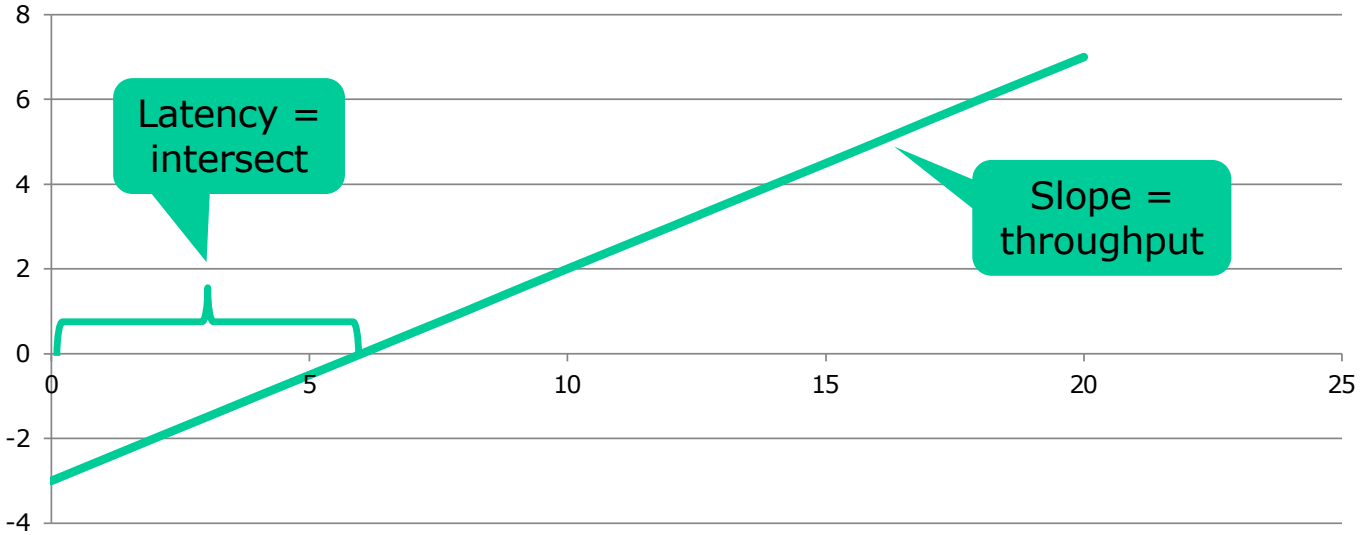

Memory Latency vs Cycle Length for Different CPUs
Aarch64
Core i9
Xeon E5
AMD7742
AMD7402P
AMD7313
Same data but
log scale x-axis
Apple M3 pro has
128KB L1 data cache
Apple M3 pro has
16MB L2 cache
Latency (ns per access)
Cycle Length (bytes) (log scale)
30 KB
100 KB
10 MB

Benchmarking mistakes and pitfalls

- Windows Upgrade etc may ruin measurements
  - Runs at unpredictable times, and is slow
- Some CPUs have a temporary "turbo mode"
  - May increase clock speed, will ruin measurements
- Some CPUs do "thermal throttling" if too hot
  - May reduce clock speed, will ruin measurements
- Measure the right thing
  - Eg when measuring binary search, do not search for the same item repeatedly (notes § 11)
- Beware of irrelevant overheads
  - For instance random number generation
  - (But now you know how to measure the overhead!)

Throughput versus latency

- Throughput is results per second
- Latency is time to first result



- Water pipe analogy:
  - Pipe diameter determines throughput, drops/sec
  - Pipe length determines latency, time to first drop of water
- We measure inverse throughput, sec/result

Professional benchmarking frameworks

- Java Microbenchmark Harness (JMH)
  - Install via a Maven project, `pom.xml` is outdated, must adapt
- BenchmarkingDotNet (BDN) for C# etc
- Annotate the methods to measure execution time

- Very useful in real SW projects, but unclear:
  - how they work and how reliable the results are
  - how to do scalability measurements (Mark8Setup)
- Same advice as for our simple framework, eg
  - "blackhole" results so JIT does not remove code
  - Use command line, use mains power, shut down other processes, etc

Reading and exercises

- Read *Microbenchmarks in Java, C# and C* (August 2026) sections 1-4, 7, 9-11.
- Read lecture notes section 9.1: *Platform threads, virtual threads and futures* (also for next week)

- Exercise sheet exercises08.pdf

- Next weeks:
  Week 09: Scalability, CPU and RAM, caches, locks, CAS
  Week 10: Java data streams and functional programming
  Week 11: Programming Graphics Processing Units (GPUs)